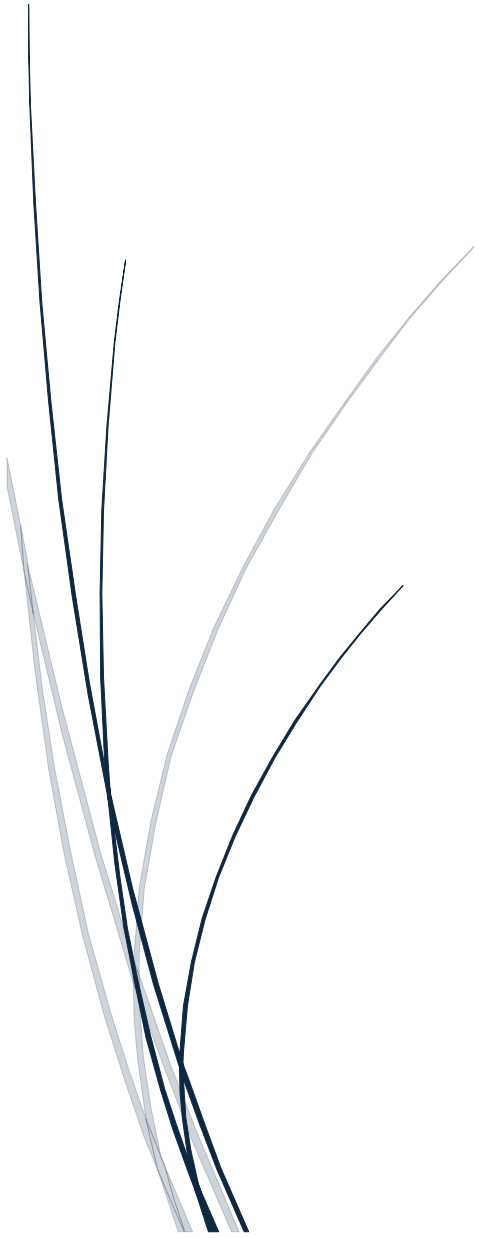
Kristiania



PG4200 – Algorithms and Data Structure

Spring 2026

23.03-21.04

Candidates – 25, 5

Group - 14

Content

1.Introduction 3
2. Implementation Dataset Setup 4
2.1 Description of Dataset 4
2.2 Data Representation in Java 4
2.2.1 Data Structure 4
2.2.2 WineReader 5
2.2.3 Timer 6
2.2.4 Printer 8
3. BubbleSort 9
3.1 Approach and Implementation 9
3.1.2 Optimised vs Non-Optimised 9
3.2 Explanation of code 10
3.3 Performance Analysis 13
3.4 Evaluation 13
4. Insertion Sort 14
4.1 Approach and Implementation 14
4.2 Explanation of code 14
4.3 Performance Analysis 18
4.4 Evaluation 19
5. Merge Sort 20
5.1 Approach and Implementation 20
5.2 Explanation of Code 21
5.3 Performance Analysis 22
5.4 Evaluation 24
6. QuickSort 25
6.1 Approach and Implementation 25
6.2 Explanation of code 25
6.3 Comparison Counts (Results) 28
6.4 Peformance Analysis 29
6.5 Evaluation 30
7. SortingAlgorithms 30
8. Conclusion 32
9. Reference List 34

1.Introduction

In our exam assignment in Algorithms and Data Structures, we will base our work on a dataset that contains information about different wine samples. This dataset includes both red and white wines, where each wine is described through several chemical properties such as acidity levels, sugar content, alcohol percentage, and a quality rating.

To solve this assignment in the best and most creative way, we have chosen to use the Java programming language. The choice of Java is based on the fact that we have worked extensively with it throughout this course, and it provides good opportunities for implementing and testing different algorithms in a structured way.

Java is a platform independent and object-oriented programming language designed to be flexible, robust, and secure in software development (Oracle, n.d.-a). The language is built around concepts such as classes and objects, which form the fundamental building blocks of java programs and make it possible to structure code in an efficient and organized way (Oracle, n.d.-b). Java also provides a comprehensive standard library with built in functions, utilities, and data structures (Oracle, n.d.-c). Which supports the development of algorithms and complex systems.

The purpose of this assignment is to examine how different sorting algorithms work in practice. The dataset provides a solid foundation for this as it contains many elements that can be sorted based on different criteria, such as quality or alcohol content.

In our assignment, several well-known sorting algorithms were used, including Bubble Sort, Insertion Sort, Merge Sort and Quick Sort. These algorithms use different strategies to sort data, making it possible to compare how they perform under various conditions.

Through this assignment, we aim to gain a better understanding of how sorting algorithms are structured, how they can be implemented in Java, and what differences exist in terms of efficiency and usage.

2. Implementation Dataset Setup

2.1 Description of Dataset

The dataset used in this assignment is the Wine Quality dataset, obtained from the UCI Machine Learning Repository (Cortez, P., 2009). The data set is publicly available at: Wine Quality - UCI Machine Learning Repository. It consists of two separate CSV files, one containing red wine and one containing white wine. Each row represents a single wine sample. The file *winequality.names* contains metadata about the dataset, including information about its origin, structure and attributes. It explains how the data was collected, explains the meaning of each variable, and provides references to the original research study (Cortez, P., 2009). Each wine sample is described by several physicochemical attributes, including fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, sulfur dioxide levels, density, PH, sulphates, and alcohol content. In addition, each sample is assigned a quality score, which serves as an overall evaluation of the wine.

The dataset is well suited for algorithmic analysis, as it contains a large number of entries and multiple numerical attributes that can be used for sorting and comparison. In this assignment, the dataset is used as input for implementing and evaluating different sorting algorithms.

2.2 Data Representation in Java

2.2.1 DataStore

In this assignment, the dataset is represented in Java using a simple and efficient data structure based on arrays of numerical values. Instead of sorting full wine objects with all attributes, we focused specifically on the alcohol values from the dataset.

The alcohol values are stored in a *double[]* array, which allows for fast access and efficient sorting. These values are loaded once and stored in a central class, *Datastore,* which makes them easily accessible throughout the program.

```java
public class DataStore {  8 usages
    public static final double[] values;

    static {
        try {
            values = WineLoader.loadAlcoholValues(
                    redPath: "wine+quality/winequality-red.csv",
                    whitePath: "wine+quality/winequality-white.csv"
            );
        } catch (Exception e) {

            throw new RuntimeException(e);
        }
    }
}
```

Figure 1 – DataStore class loading alcohol values

Each element in the array represents the alcohol content of a single wine sample. This means that every row in the dataset is transformed into numerical value that can be used directly by sorting algorithms.

This approach simplifies the implementation of algorithms, as they only need to handle numerical data instead of complex objects. It also improves performance, since arrays provide faster access and lower overhead compared to more complex data structures.

By representing the dataset in this way, it becomes easier to apply and compare different sorting algorithms, as all algorithms operate in the same standardized data format.

2.2.2 WineReader

The *WineReader* class is responsible for reading alcohol values from the red and white wine CSV files used in assignment. Its main purpose is to extract the alcohol column from both datasets and return all unique values in a *double[]* array.

The class contains a public method, *loadAlcoholValues(),* which takes the file paths for the red and white wine CSV files. It uses a *HashSet<Double>* to store the alcohol values,

ensuring that duplicates are automatically removed. The method calls the private helper method *loadFile()* for each file.

The *loadFile()* method reads CSV file line by line using a *BufferedReader*. The first line is skipped because it contains the header. Each remaining line is split using a semicolon, and the alcohol value is extracted from column index 10 (the 11th column in the dataset). The value is parsed as a *double* and added to the set.

After both files have been processed, the set of unique alcohol value is converted into a *double[]* array and returned. This results in a clean and compact dataset that can be used directly by the insertion Sort algorithm in the next part of the assignment.

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.util.HashSet;
import java.util.Set;

public class WineReader {  1 usage
    //Loads unique alchol values from both red and white CSV files.
    public static double[] loadAlcoholValues(String redPath, String whitePath) throws Exception {  1 usage
        Set<Double> alcoholSet = new HashSet<>();

        loadFile(redPath, alcoholSet);  //Read red whine file
        loadFile(whitePath, alcoholSet); //read white wine file
        //Convert set<Double> to doble[]
        double[] result = new double[alcoholSet.size()];
        int i = 0;
        for (double value : alcoholSet) {
            result[i++] = value;
        }
        return result;
    }
    //Reads a single CSV file and extracts alcohol values
    private static void loadFile(String path, Set<Double> alcoholSet) throws Exception {  2 usages
        BufferedReader br = new BufferedReader(new FileReader(path));
        String line;

        br.readLine(); // Skip header line

        while ((line = br.readLine()) != null) {
            String[] parts = line.split( regex: ";");
            double alcohol = Double.parseDouble(parts[10]); // kolonne 11 = alcohol
            alcoholSet.add(alcohol);
        }

        br.close(); // Close file reader
    }
}
```

Figure 2: Implementation of the WineReader class

2.2.3 Timer

The *Timer* class is used to measure the execution time of the sorting algorithms. It provides a simple and efficient way to record how long the different assignments takes to run on the dataset.

The class works by storing two timestaps, one at the start of the operation and one at the end. The *begin()* method records the current time using *System.nanoTime(),* which returns the

current time in nanoseconds. After the sorting process is completed, the *finish()* method is called to record the end time. The total execution time is calculated by subtracting the start time from the end time.

```
public class Timer {  22 usages

    private long start;  4 usages
    private long end;  4 usages

    public void begin() {  12 usages
        start = System.nanoTime();
    }

    public void finish() {  12 usages
        end = System.nanoTime();
    }
```

Figure 3 – Timer class methods for recording start and end time

The class provides three methods for retrieving the measured time in different units. The *getNanoTime()* method the duration in nanoseconds, which is the most precise measurement. The *getMicroTime()* method converts the result into microseconds by dividing by 1000, while *getMilliTime()* method converts it into milliseconds by dividing by 1 000 000.

In this implementation, the *getMicroTime()* method is used in the main program to display the execution time in microseconds. This offers a good balance between precision and readability, making it easier to compare performance results.

```
    public long getNanoTime() {  no usages
        return end - start;
    }

    public long getMicroTime() {  12 usages
        return (end - start) / 1000;
    }

    public long getMilliTime() {  no usages
        return (end - start) / 1_000_000;
    }
}
```

Figure 4 – converting execution time into different units

2.2.4 Printer

The *Printer* class is used to display the contents of the dataset before and after sorting. It is implemented as a utility class with static methods that support both primitive array *double[]* and generic *ArrayList* structures.

The class includes a parameter, *limit* that controls how many elements are printed. If the limit is set to 0, the entire dataset is printed. Otherwise, only a specified number of elements from the beginning of the dataset are displayed. This makes it possible to inspect the data without overwhelming the output when working with large datasets.

By supporting both arrays and ArrayList, the class becomes flexible reusable for different data types, not only *Double* values. This improves the overall design and reusability of the code.

Although the *Printer* class does not influence the performance of the sorting algorithm, it plays an important role in debugging and validating the results by providing a clear and controlled way to visualize the dataset.

```java
public class Printer {  3 usages

    public static void printArray(double[] arr, int limit) {  1 usage
        if (limit == 0) {
            for (double v : arr) {
                System.out.println(v);
            }
        } else {
            for (int i = 0; i < limit && i < arr.length; i++) {
                System.out.println(arr[i]);
            }
        }
    }

    public static <T> void printArray(java.util.ArrayList<T> list, int limit) {  2 usages
        if (limit == 0) {
            for (T v : list) {
                System.out.println(v);
            }
        } else {
            for (int i = 0; i < limit && i < list.size(); i++) {
                System.out.println(list.get(i));
            }
        }
    }
}
```

Figure 5 – Utility class for printing dataset values before and after sorting

3. BubbleSort

3.1 Approach and Implementation

The goal of this task was to sort the unique alcohol content values taken from the combined red and white wine datasets. To achieve this, two versions of the Bubble Sort algorithm were implemented, a non-optimised version and an optimised version. Both algorithms were applied to identical copies of the dataset to ensure a fair comparison.

The Bubble Sort algorithm works by repeatedly comparing neighbouring elements and swapping them if they are in the wrong order (Alake, 2025). Over multiple passes through the list, the largest remaining value gradually moves toward the end, which is why the method is described as “bubbling” the values into place (Alake, 2025).

The process continues until the entire list is sorted in ascending order. Bubble Sort is simple to understand and implement, but it is known for being slow on larger datasets because it performs many repeated comparisons (Alake, 2025). Its general time complexity is quadratic, meaning that the number of operations increases rapidly as the dataset grows (Alake, 2025).

3.1.1 Optimised vs Non-Optimised

The non-optimised version of Bubble Sort always performs the same number of comparisons, even if the list is already sorted (Alake, 2025). It completes a full set of passes through the array, resulting in a time complexity of $O(n^2)$ in all case. The optimised version improves this by introducing a swap-detection mechanism. During each pass, the algorithm checks whether any swaps were made (Alake, 2025). If a full pass completes without any swaps, the algorithm stops early because the list is already sorted. This allows the optimised version to run significantly faster when the input is already sorted or nearly sorted, giving it a best-case time complexity of $O(n)$, while the worst case remains $O(n^2)$ (Alake, 2025).

3.2 Explanation of code

```java
import java.util.Random;

public class BubbleSort {

    public static void main(String[] args) throws Exception {

        double[] values = DataStore.values.clone();

        double[] arr1 = values.clone();
        double[] arr2 = values.clone();

        Timer timer = new Timer();


        timer.begin();
        bubbleSort(arr1);
        timer.finish();
        System.out.println("Non-optimised Bubble Sort time (original): " + timer.getMicroTime() + " µs");

        timer.begin();
        bubbleSortOptimised(arr2);
        timer.finish();
        System.out.println("Optimised Bubble Sort time (original): " + timer.getMicroTime() + " µs");
```

Figure 6 – Main method running both Bubble Sort versions

The program begins with the main method by loading the alcohol values from *DataStore.values* and cloning them. Cloning is important because it ensures that both sorting algorithms receive exactly the same input. Two arrays, *arr1* and *arr2*, are created so that the non-optimised and optimised Bubble Sort methods can be tested independently.

A *Timer* object is then created to measure how long each sorting method takes to complete. The program first measures the execution time of the non-optimised Bubble Sort by starting the timer, running the method on *arr1*, and stopping the timer. The time is printed in microseconds. The same steps are repeated for the optimised version using *arr2*, allowing a direct comparison of the two algorithms on the original dataset.

```
    double[] shuffled1 = values.clone();
    double[] shuffled2 = values.clone();

    shuffleArray(shuffled1);
    shuffleArray(shuffled2);

    timer.begin();
    bubbleSort(shuffled1);
    timer.finish();
    System.out.println("Non-optimised Bubble Sort time (shuffled): " + timer.getMicroTime() + " µs");

    timer.begin();
    bubbleSortOptimised(shuffled2);
    timer.finish();
    System.out.println("Optimised Bubble Sort time (shuffled): " + timer.getMicroTime() + " µs");

    System.out.println("\nSorted values (Non-optimised):");
    Printer.printArray(arr1, limit: 0);

    System.out.println("\nSorted values (Optimised):");
    Printer.printArray(arr2, limit: 0);
}
```

Figure 7 – Bubble Sort on shuffled data

To analyse how the order of the input effects performance, the program creates two additional arrays, *shuffled1* and *shuffled2*, which are also clones of the original values. These arrays are passed by the *shuffleArray* method. This method uses the Fisher–Yates algorithm, which iterates through the array from the end to the beginning and swaps each element with another element at a random index. This produces a fully randomised order. After shuffling, the program again measures the sorting time again for both algorithms but now using random data. At the end of the main method, the program prints the sorted values from both versions to demonstrate that they produce the same correct output.

```
public static void shuffleArray(double[] arr) {  2 usages
    Random random = new Random();

    for (int i = arr.length - 1; i > 0; i--) {
        int j = random.nextInt( bound: i + 1);

        double temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Figure 8 – Shuffling array using Fisher-Yates

The *shuffleArray* method is responsible for randomising the order of the values. It creates a *Random* object and loops from the last index down to the first. At each step, it selects a

random index and swaps the two elements. This ensures that the dataset is thoroughly shuffled before sorting begins (GeeksforGeeks, 2025a)

```java
public static void bubbleSort(double[] arr) {  3 usages
    int n = arr.length;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                double temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Figure 9 – Basic Bubble Sort implementation

The *bubbleSort* method is the basic, non-optimised version of Bubble Sort. It uses two nested loops that always run through the entire array. During each pass, it compares two neighbouring values and swaps them if they are in the wrong order. Because the loops always run the full number of iterations, the method performs the same amount of work even when the array is already sorted. This makes the method simple but inefficient.

```java
public static void bubbleSortOptimised(double[] arr) {  3 usages
    int n = arr.length;
    for (int i = 0; i < n; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                double temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) {
            break;
        }
    }
}
```

Figure 10 – Optimised Bubble Sort with early exit

The *bubbleSortOptimised* method improves on the basic version by introducing a boolean variable called *swapped*. At the start of each pass, the variable is set to *false*. If any swap occurs during the pass, the variable becomes *true*. If a full pass completes without any swaps, the method terminates early because the array is already sorted. This optimization makes the

algorithm significantly faster when the input is already sorted or nearly sorted. Additionally, the inner loop becomes shorter with each pass because the largest values gradually move to the end of the array and no longer need to be checked.

3.3 Performance Analysis

```
Non-optimised Bubble Sort time (original): 370 µs
Optimised Bubble Sort time (original): 243 µs
Non-optimised Bubble Sort time (shuffled): 307 µs
Optimised Bubble Sort time (shuffled): 247 µs
```

Figure 11 – Runtime comparison for Bubble Sort (original vs shuffled)

The results show a performance difference between the non-optimised and optimised Bubble Sort algorithms. On the original dataset, the non optimised version required 370 µs, while the optimised version completed the sort in 243 µs. This is because the optimised version can stop early when a pass completes with no swaps, whereas the non-optimised version always performs the full set of comparisons.

When the list randomly shuffled, the non-optimised version took 307 µs, which is close to its time on the original data. This is expected, since it performs the same amount of work regardless of input order. The optimised version took 247 µs on the shuffled list, which is also reasonable because random input usually gives it less opportunity to stop early.

Although the measured times change slightly, the theoretical time complexity remains the same. The non-optimised Bubble Sort always runs in $O(n^2)$, while the optimised version has a best-case complexity of $O(n)$ but still $O(n^2)$ in the average and worst case. Shuffling the list does not change these complexity classes. It only affects how quickly the optimised version can finish

3.4 Evaluation

The results demonstrate that the optimised Bubble Sort performs better in practice than the non-optimised version. On the original dataset, the optimised version was faster because it could stop early when the list was already partially ordered. In contrast, the non-optimised version always runs the full number of comparisons and therefore cannot take advantage of any existing order in the data.

When the list was shuffled, the optimised version became slightly slower, while the non-optimised version showed almost no change in performance. This makes sense because the optimised version requires more passes when the list is in a random order. Even though the times changed, both algorithms still produced the same correctly sorted output.

Overall, the results show that the optimised Bubble Sort is faster in practice, especially when the input is already ordered or partly ordered. The non-optimised version behaves the same in every situation, while the optimised version is more sensitive to the structure of the input. This means that the optimised version gives better performance in real-world scenarios where data is often not completely random.

4. Insertion Sort

4.1 Approach and Implementation

Insertion Sort is a comparison-based sorting algorithm that builds a sorted sequence incrementally (Sedgewick & Wayne, 2011, p.250). The algorithm iterates through the dataset and inserts each element into its correct position within a growing sorted sublist (Sedwewick & Wayne, 2011, p.250).

The first element is considered sorted by default. For each subsequent element, the algorithm compares it with elements in the sorted portion and shifts larger elements one position to the right until the correct position is found. The element is then inserted into this position. This process continues until all elements have been processed, resulting in a fully sorted dataset.

4.2 Explanation of code

In our implementation, the dataset is initialized by loading alcohol values from the *DataStore* and converting them into an *ArrayList<Double*>. This structure allows flexible access and modification of elements during sorting. A *timer* object is used to measure the execution time of the algorithm, ensuring that performance can be evaluated accurately. The sorting process is executed by calling the *insertionSort* method, and the total runtime is recorded in microseconds.

```
public static void main(String[] args) throws Exception {

    ArrayList<Double> alcoholValues = loadAsList();

    Timer timer = new Timer();

    timer.begin();
    insertionSort(alcoholValues);
    timer.finish();
```

Figure 12 – Main method initializing dataset, starting timer, and executing insertion.

This setup ensures that both correctness and performance of the algorithm can be tested in a controlled and repeatable manner.

The *main* method has been extended to perform two separate tests to evaluate the algorithm under different input conditions. First, the dataset is sorted in its original order, and the execution time is measured. Then, the list is shuffled using *Collections.shuffle()* to create a random input order, after which the sorting algorithm is executed again. This allows for a direct comparison of how insertion sort performs on already sorted versus randomly ordered data.

```
    Collections.shuffle(alcoholValues);

    timer.begin();
    insertionSort(alcoholValues);
    timer.finish();

    System.out.println("Time (shuffled): "
            + timer.getMicroTime() + " µs");

    System.out.println("\nAll alcohol values sorted:");
    Printer.printArray(alcoholValues, limit: 0);
}
```

Figure 13 -Shuffling dataset and re-running insertion sort to measure performance on random input.

The dataset is converted from a *double[]* array into an *ArrayList<Double>*. The original data is cloned from the *DataStore* to avoid modifying the source dataset. Each value is the iterated over and added to the *ArrayList*, creating a dynamic data structure that is more suitable for sorting operations. This conversion is essential because the Insertion Sort algorithm relies on shifting elements, which is easier to implement and manage using an *ArrayList* compared to a fixed-size array.

```
private static ArrayList<Double> loadAsList() throws Exception {
    double[] values = DataStore.values.clone();

    ArrayList<Double> list = new ArrayList<>();
    for (double v : values) {
        list.add(v);
    }
    return list;
```

Figure 14 – Converting array data from DataStore into an ArrayList for sorting.

The core implementation of the insertion sort algorithm begins at index 1, since the first element is already considered sorted. For each iteration, the current value is stored in a temporary variable to prevent it from being overwritten during the shifting process. The inner loop compares the current value with elements in the sorted portion of the list.

```
public static void insertionSort(ArrayList<Double> values) {
    for (int i = 1; i < values.size(); i++) {
        double currentValue = values.get(i);
        int j = i;

        while (j > 0 && values.get(j - 1) > currentValue) {
            values.set(j, values.get(j - 1));
            j--;
        }

        values.set(j, currentValue);
    }
}
```

Figure 15 – Implementation of the insertion sort algorithm with element shifting.

If a larger element is found, it is shifted to the right. This continues until the correct position for the current value is identified. The value is then inserted into this position, ensuring that the sorted sub-list remains ordered after each iteration.

```java
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        /* Move elements of arr[0..i-1], that
are
           greater than key, to one position
ahead
           of their current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Figure 16 – Example implementation of the Insertion Sort algorithm in Java. (GeeksforGeeks, 2026b)

This implementation closely follows the theoretical behavior of insertion sort, where elements are inserted into their correct position rather than swapped.

We have also implemented a validation method to verify that the dataset is correctly sorted after execution. The method iterates through the list and checks whether each element is less than or equal to the next.

```java
public static boolean isSorted(ArrayList<Double> values) {
    for (int i = 1; i < values.size(); i++) {
        if (values.get(i - 1) > values.get(i)) {
            return false;
        }
    }
    return true;
}
```

Figure 17 – Validation method used to verify that the dataset is correctly sorted.

If any pair of elements is found to be out of order, the method returns *false*, indicating an error in the sorting process. Otherwise, it confirms that the dataset is correctly sorted. This additional verification step improves the reliability of the implementation and ensures that the algorithm produces correct results.

4.3 Performance Analysis

The time complexity of insertion sort in this implementation can be understood by analyzing the loops in the *insertionSort* method. The outer *for* loop runs through the entire list once, starting from index 1, meaning that every element in the dataset is processed.

```
public static void insertionSort(ArrayList<Double> values) {  2 usa
    for (int i = 1; i < values.size(); i++) {
        double currentValue = values.get(i);
        int j = i;

        while (j > 0 && values.get(j - 1) > currentValue) {
            values.set(j, values.get(j - 1));
            j--;
        }

        values.set(j, currentValue);
    }
}
```

Figure 18 – Insertion sort algorithm logic

Inside the outer loop, the *while* loop compares the current value with the elements in the sorted portion of the list. If the previous elements are larger than the current value, they are shifted from one position to the right. This continues until the correct position for the current value is found.

```
Time it took to sort: 782 µs (micro seconds)
```

Figure 19 – Runtime for sorted data

In the best-case scenario, the dataset is already sorted. In this case, the condition in the *while* loop fails immediately for every iteration. The algorithm simply iterates through the outer loop once, giving a best-case time complexity of O(n).

In the worst-case scenario, the dataset is sorted in reverse order. Each element must then be compared with all previously sorted elements and shifted to the correct position. This causes the inner loop to run many times for each iteration of the outer loop, resulting in a worst-case time complexity of $O(n^2)$.

The average-case complexity is also $O(n^2)$, since a randomly ordered dataset typically requires multiple comparisons and shifts before each element reaches its correct position.

The order of the input data therefore has a direct impact on the performance of insertion sort. A nearly sorted dataset results in very few comparisons and no shifts, giving O(n) performance. A randomly ordered dataset requires significantly more comparisons and shifts resulting in $O(n^2)$ behavior. In the worst-case, where the dataset is reversed, every element must be compared with all previous elements producing the maximum number of operations and a time complexity of $O(n^2)$.

Shuffling the data before sorting will generally make the algorithm slower compared to sorting an already sorted dataset, as it increases the number of required comparisons and shifts. This demonstrates that Insertion Sort is efficient for nearly sorted data, but less efficient for randomly ordered or reversed datasets.

To further analyze the impact of input order, the dataset was shuffled using *Collections.shuffle()* before running the algorithm a second time. This ensures that the data is randomly ordered, representing the average-case scenario. The measured execution time after shuffling was higher than for the already sorted dataset. This confirms that insertion sort performs faster on a nearly sorted dataset and slower on randomly ordered data due to the increased number of operations. However, the theoretical time complexity remains unchanged at $O(n^2)$ in the average case.

```
Time it took to sort: 782 µs (micro seconds)
Time (shuffled): 599 µs (micro seconds)
```

Figure 20 – Runtime comparison between sorted and shuffled data

Insertion Sort generally performs better than Bubble Sort in practice because it requires fewer operations, especially for nearly sorted datasets. However, both algorithms have a time complexity of $O(n^2)$ in the average and worst cases.

4.4 Evaluation

Insertion sort performs well in scenarios where the dataset is small or nearly sorted. In such cases, the algorithm requires very few comparisons and minimal shifting of elements, resulting in efficient performance close to O(n). This makes Insertion Sort a practical choice when working with datasets that are already partially ordered or when only few elements are out of place.

However, the algorithm performs poorly on large or randomly ordered datasets. In these situations, many comparisons and shifts are required for each element, leading to a time complexity of $O(n^2)$. This significantly increases the running time and makes Insertion Sort inefficient compared to more advanced algorithms such as Merge Sort or Quicksort, which offer O(n log n) performance in most cases.

The results from implementation clearly show that the performance of a sorting algorithm is highly dependent on the structure of the data input. Although Insertion Sort is simple and easy to implement, its efficiency varies greatly depending on whether the data is sorted, random or reversed.

Overall, this work demonstrates the importance of selecting an appropriate algorithm based on the characteristics of the dataset. Insertion sort is suitable choice for small or nearly sorted datasets, but it is not recommended for larger datasets, where more efficient algorithms should be used.

5. Merge Sort

5.1 Approach and Implementation

The goal of this implementation is to sort the unique alcohol-content values extracted from combined datasets. Duplicates values are removed using a *HashSet*, after which the remaining values are stored in an *ArrayList* for sorting.

Merge Sort is used as the sorting algorithm. It follows a divide and conquer strategy by recursively splitting the dataset into smaller sublists until each sublist contains a single element. (Sedgewick & Wayne, 2011, p.270) These sublists are then merged back together in ascending order. During the merge phase, two sorted sublists are compared element by element, and the smallest available value is placed into a temporary list is copied back into the original list. Although this implementation uses an *ArrayList* instead of traditional array, the core logic of Merge Sort remains unchanged. The algorithm still divides the dataset, sorts each part recursively, and merges the results into fully ordered list. (Sedgewick & Wayne, 2011, p.271)

Merge sort works by applying a divide and conquer strategy to sort the dataset. (Sedgewick & Wayne, 2011, p.277) The algorithm begins by dividing the list into two halves. This process continues recursively until each sublist contains only one element, which is inherently sorted.

Once the dataset has been divided into single element sublists, the algorithm starts merging them back together. During this merge phase, two sorted sublists are combined into a single sorted list by comparing their elements. The smallest element from the two sublists is selected and added to a temporary list, and this process continues until all elements have been merged.

This merging process ensures that the resulting list remains sorted at every step. By repeatedly dividing and merging the dataset, Merge Sort efficiently produces a fully sorted list in ascending order.

5.2 Explanation of Code

The *mergeSort* method takes three parameters: *arr*, *left*, and *right*. The parameter *arr* represents the *ArrayList* containing the alcohol values to be sorted. The parameters *left* and *right* define the current section of the list that is being processed. In other words, they indicate the starting and ending indices of the sublist that should be divided and sorted.

```
public static void mergeSort(ArrayList<Double> arr, int left, int right)
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr,  left: mid + 1, right);

        merge(arr, left, mid, right);
    }
}
```

Figure 21 – Splitting the dataset using merge sort

The variable *mid* is used to determine the midpoint of the current section of the list. It is calculated as *(left + right) / 2*, which makes it possible to split the list into two smaller parts, the left half from *left* to *mid*, and the right half from *mid + 1* to *right*.

The *mergeSort* method is called recursively because Merge Sort operates by repeatedly dividing the dataset into smaller sublists. The recursion continues with only one element. A single- elements list inherently sorted, so this forms the base case of the algorithm.

After the recursive division is complete, the *merge()* method is used to combine the smaller sorted sublists into larger sorted sections. This method takes the two sorted halves and merges them into the correct order.

```
while (i <= mid && j <= right) {
    if (arr.get(i) <= arr.get(j)) {
        temp.add(arr.get(i++));
    } else {
        temp.add(arr.get(j++));
    }
}
```

Figure 22 – Comparing and merging elements

Inside *merge(),* a temporary list called *temp* is used to store the merged values. This temporary structure is necessary because it allows the algorithm to build the sorted result without overwriting values in the original list while comparisons are still being made. Once the merge is complete, the contents of *temp* are copied back into the appropriate section of the original list.

```
ArrayList<Double> temp = new ArrayList<>();
```

Figure 23 – Temporary storage for merged values

The actual merge process is carried out using two index variables, *I* and *j*. The variable *I* starts at the beginning of the left half, while *j* starts at the beginning of the right half. The algorithm compares the values at these positions and adds the smaller one to the temporary list, *temp*. If the value in the left half is smaller, it is added to *temp* and *I* is incremented. Otherwise, the value from the right half is added and *j* is incremented. This process continues until one of the halves has no remaining elements.

When one half has been fully processed, the remaining elements from the other half are copied into *temp*. At this point, *temp* contains all values from the current section in sorted order. Finally, the values from *temp* are copied back into the original list *arr*, starting at index *left*. In this way, the algorithm gradually builds a fully sorted list by merging smaller sorted sections into larger ones.

5.3 Performance Analysis

Merge sort has a time complexity of O(n log n) in the best, worst and average case. This means that the performance of the algorithm remains consistent regardless of the input order. The reason is that Merge Sort always follows the same procedure, it divides the dataset into smaller sublists and then merges them back together. The number of divisions and merge operations depends solely on the size of the dataset, not on how the data is arranged. Unlike algorithms such as Bubble Sort and Insertion Sort, Merge Sort does not benefit from already sorted data, not does it perform worse on reversed data. The algorithm always performs the same number of steps because it consistently divides and merges the data in the same pattern.

```java
public static void mergeSort(ArrayList<Double> arr, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, left: mid + 1, right);

        merge(arr, left, mid, right);
    }
}
```

Figure 24 – MergeSort method

However, Merge Sort requires additional memory. During the merge step, a temporary list is used to store intermediate results before copying them back into the original list. This results in a space complexity of O(n), since extra memory proportional to the dataset size is required.

```java
ArrayList<Double> temp = new ArrayList<>();
```

Figure 25 – Temporary list used during merging, showing additional memory usage

A merge operation refers to each time the *merge()* method is called to combine two sorted sublists into single sorted list. In this implementation, the number of merge operations is tracked using the variable *mergeCount*, which is increased by one every time the *merge()* method is executed. The total number of merge operations is determined by the recursive structure of the algorithm. Since Merge Sort always splits the dataset in the same way, the number of merge operations remains the same regardless of input order. This is confirmed by the program results, where both the original dataset and the shuffled dataset produced the same number of merge operations. This behavior reflects the deterministic nature of the algorithm, *merge()* is called once for every split that must be merged back together.

```java
public class MergeSort {

    static int mergeCount = 0;
```

Figure 26 - Track the number of merge operations

```java
mergeCount++;
```

Figure 27 – MergeCount for each merge operation

To analyze the impact of input order, the dataset was shuffled using *Collections.shuffle()* before running the algorithm a second time. This created a randomly ordered dataset representing the average-case scenario. The results show that shuffling the data does not significantly affect execution time or the number of merge operations. This is because Merge Sort always divides and merges the dataset in the

same way, regardless of input order. This behavior contrasts with algorithms such as Bubble Sort and Insertion Sort, which depend heavily on the initial order of the data. For example, Insertion Sort performs very efficiently on nearly sorted data, but much slower on randomly ordered or reversed data. Merge sort, on the other hand, shows consistent in all cases, making it predictable and reliable for large dataset.

```
Collections.shuffle(shuffledValues);
```

Figure 28 – Shuffling dataset to test impact of input order

```
MergeSort time (original order): 1116 µs (micro seconds)
Number of merge operations (original order): 110

MergeSort time (shuffled order): 324 µs (micro seconds)
Number of merge operations (shuffled order): 110
```

Figure 29 – Execution time and merge operations for original vs shuffled dataset

5.4 Evaluation

Merge sort is more efficient than algorithms such as Bubble Sort and Insertion Sort when working with large datasets. This is due to its consistent time complexity of O(n log n) in all cases, while the other algorithms can degrade to $O(n^2)$ in the worst case. As a result, Merge Sort is better suited for handling larger amounts of data.

A key advantage of Merge Sort is that it performs the same number of operations regardless of input order. The results from this implementation show that the number of merge operations is identical for both the original and shuffled dataset. This makes the algorithm predictable and consistent, which is valuable when analyzing performance.

Merge sort is also a stable sorting algorithm, meaning that equal values retain their original order after sorting. This can be important in applications where the relative order of equal elements carries meaning.

Merge Sort has limitations. It requires additional memory due to the temporary list used during the merge phase, resulting in a time complexity of O(n). The recursive structure also introduces overhead, which can make Merge Sort less efficient for small datasets compared to simpler algorithms like Insertion Sort.

Overall, merge sort is a strong choice for larger datasets where consistent performance and stability are important, while simpler algorithms may be more suitable for smaller datasets or cases where memory usage must be minimized.

6. QuickSort

6.1 Approach and Implementation

QuickSort sorts a list by selecting a pivot element and dividing the remaining values into two groups, those smaller than the pivot and those larger than the pivot. (Sedgewick & Wayne, 2011, p.291). This process, known as partitioning, creates two sublists that can be sorted independently. QuickSort then applies the same procedure recursively to each sublist until the entire list is ordered.

The choice of pivot plays a significant role in performance. (Sedgewick & Wayne, 2011, p.294). A well-chosen pivot produces two sublists of similar size, allowing the algorithm to run efficiently. A poorly chosen pivot results in highly unbalanced partitions, which increases the number of comparisons and slows down the sorting process. (Sedgewick & Wayne, 2011, p.295) Because of this, different pivot-selection strategies can lead to different performance outcomes, even though the underlying algorithm remains the same.

6.2 Explanation of code

This task contains four different versions of the QuickSort algorithm. Each version uses the same general structure, but they choose the pivot in different ways. All versions clone the dataset, call the QuickSort method, count the number of comparisons inside the partition step, and finally print the number of comparisons and the first ten sorted values. The goal is to compare how the pivot choice affects the number of comparisons.

The QuickSort algorithm works by choosing a pivot, splitting the array into two parts (values smaller than the pivot and values larger than the pivot), and then sorting each part using recursion. The partition method is responsible for moving values to the correct side of the pivot. Every time the algorithm compares a value with the pivot, the comparison counter increases.

```java
public static int partition(double[] arr, int low, int high) {  1 usage
    double pivot = arr[low];
    int leftIndex = low;

    for (int rightIndex = low + 1; rightIndex <= high; rightIndex++) {
        comparisonCount++;

        if (arr[rightIndex] < pivot) {
            leftIndex++;
            swap(arr, leftIndex, rightIndex);
        }
    }

    swap(arr, low, leftIndex);
    return leftIndex;
}
```

Figure 30 - QuickSort_A first element pivot

In this version, the first element of the current subarray is chosen as the pivot. The algorithm compares the remaining elements with the pivot and moves all smaller values to the left side. After the scan, the pivot is swapped into its final position. This approach is simple, but it can lead to unbalanced partitions when the first element is a poor pivot choice, such as in already sorted or nearly sorted data. (GeeksforGeeks, 2025c)

```java
public static int partition(double[] arr, int low, int high) {  1 usage
    double pivot = arr[high];
    int leftIndex = low - 1;

    for (int rightIndex = low; rightIndex < high; rightIndex++) {
        comparisonCount++;

        if (arr[rightIndex] < pivot) {
            leftIndex++;
            swap(arr, leftIndex, rightIndex);
        }
    }

    swap(arr, i: leftIndex + 1, high);
    return leftIndex + 1;
}
```

Figure 31 - QuickSort_B last element pivot

The second version uses the last element as the pivot. The algorithm takes the value at the high index and compares all other elements with it. Values smaller than the pivot are moved to the left side, and after the scan the pivot is swapped into its correct position. This version is similar to the first one, but the pivot is taken from the opposite end of the array. It can also produce unbalanced partitions when the last element is a poor pivot choice, such as in already sorted or nearly sorted data.

```
public static int partition(double[] arr, int low, int high) {  1 usage
    int randomIndex = random.nextInt( bound: high - low + 1) + low;
    swap(arr, randomIndex, high);

    double pivot = arr[high];
    int leftIndex = low - 1;

    for (int rightIndex = low; rightIndex < high; rightIndex++) {
        comparisonCount++;

        if (arr[rightIndex] < pivot) {
            leftIndex++;
            swap(arr, leftIndex, rightIndex);
        }
    }

    swap(arr,  i: leftIndex + 1, high);
    return leftIndex + 1;
}
```

Figure 32 - QuickSort_C random pivot

The third version chooses a random element as the pivot. Before the partitioning begins, a random index between low and high is selected and that element is swapped with the element at high position. The value at high is then used as the pivot. The rest of the partitioning process works the same as in the previous versions. All elements smaller than the pivot are moved to the left side, and the pivot is finally placed in its correct position. This strategy helps reduce the risk of unbalanced partitions, especially when the input data is already sorted or nearly sorted.

```java
public static int partition(double[] arr, int low, int high) {  1 usage
    int medianIndex = findMedianOfThreeIndex(arr, low, high);
    swap(arr, medianIndex, high);

    double pivot = arr[high];
    int leftIndex = low - 1;

    for (int rightIndex = low; rightIndex < high; rightIndex++) {
        comparisonCount++;

        if (arr[rightIndex] < pivot) {
            leftIndex++;
            swap(arr, leftIndex, rightIndex);
        }
    }

    swap(arr, i: leftIndex + 1, high);
    return leftIndex + 1;
}

public static int findMedianOfThreeIndex(double[] arr, int low, int high) {  1 usage
    int mid = (low + high) / 2;

    double first = arr[low];
    double middle = arr[mid];
    double last = arr[high];

    if ((first <= middle && middle <= last) || (last <= middle && middle <= first)) {
        return mid;
    }

    if ((middle <= first && first <= last) || (last <= first && first <= middle)) {
        return low;
    }

    return high;
```

Figure 33 - QuickSort_D median of three pivot

In the code, the method findMedianOfThreeIndex compares the first, middle, and last values of the current subarray and returns the index of the median value. That index is then swapped with the last position, so the rest of the partition method can use it as the pivot. After that, the algorithm moves all smaller values to the values to the left and places the pivot in its final position.

6.3 Comparison Counts (Results)

```
Quick Sort - First Element Pivot
Number of comparisons: 709
Sorted correctly: true
First 10 sorted values: 8.0 8.4 8.5 8.6 8.7 8.8 8.9 9.0 9.05 9.1

Process finished with exit code 0
```

Figure 34 - first element pivot

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\algorithms and data
Quick Sort - Last Element Pivot
Number of comparisons: 747
Sorted correctly: true
First 10 sorted values: 8.0 8.4 8.5 8.6 8.7 8.8 8.9 9.0 9.05 9.1

Process finished with exit code 0
```

Figure 35 - Last element pivot

```
Quick Sort - Random Element Pivot
Number of comparisons: 795
Sorted correctly: true
First 10 sorted values: 8.0 8.4 8.5 8.6 8.7 8.8 8.9 9.0 9.05 9.1
```

Figure 36 - Random element pivot

```
Quick Sort - Median of Three Pivot
Number of comparisons: 640
Sorted correctly: true
First 10 sorted values: 8.0 8.4 8.5 8.6 8.7 8.8 8.9 9.0 9.05 9.1
```

Figure 37 – Median-of-three pivot

The number of comparisons changed depending on which the pivot was used. Using the first element as pivot resulted in 709 comparisons, the last element pivot resulted in 747 comparisons, the random element pivot resulted in 795 comparisons, and the median-of-three pivot resulted in 640 comparisons. All four versions sorted the data correctly, but they did not perform equally in terms of comparisons.

6.4 Performance Analysis

The results show that the choice of pivot has a clear effect on the practical performance of QuickSort. In this dataset, the median-of-three strategy performed best because it produced the fewest comparisons. This means it created more balanced partitions. The random pivot strategy performed worst in this particular run, but its performance can vary because the pivot is chosen randomly. The first and last element strategies performed better than the random pivot in this case, but they were less efficient than the median-of-three method.

6.5 Evaluation

The median-of-three pivot selection strategy performed best in this dataset because it produced the fewest comparisons. This suggests that it created more balanced partitions than the other strategies. The first and last elements strategies also worked correctly, but they required more comparisons which indicate that they were less effective pivot choices for this dataset. The random pivot strategy gave the highest number of comparisons in this run, although its performance may vary between executions because the pivot is chosen randomly. Overall, the result shows that pivot choice has a clear impact on the practical efficiency of QuickSort, even though all versions still sort the data correctly.

7. SortingAlgorithms

This is an additional program that compares the performance of different sorting algorithms by measuring how long they take to sort the same dataset. The algorithms tested are all four above. The goal is to see how the algorithms behave with different types of input, such as random data, already sorted data and reversed data. This makes it possible to understand how input order affects performance.

We compared the execution time of several sorting algorithms when they are applied to the same dataset in different input orders. At the start of the program, the original dataset is copied so that each algorithm receives identical input.

```java
public class SortingAlgorithms {

    public static void main(String[] args) throws Exception {
        double[] original = DataStore.values.clone();

        double[] randomData = original.clone();

        double[] sortedData = original.clone();
        Arrays.sort(sortedData);

        double[] reversedData = original.clone();
        Arrays.sort(reversedData);
        reverseArray(reversedData);
```

Figure 38 – Preparing dataset copies for fair comparison of sorting algorithms

This is done to make sure the original data is not changed. After that, three different input types are prepared. *randomData* keeps the original order, *sortedData* is sorted in ascending order using *Array.sort(),* and *reversedData* is first sorted and then reversed by the *reverseArray()* method. This makes it possible to test how each sorting algorithm performs depending on the input order.

The program then prints a table header. This output format gives a clear comparison of the execution times for each algorithm.

```java
System.out.println("Algorithm              | Random (µs) | Sorted (µs) | Reversed (µs)");
System.out.println("-----------------------------------------------------------------------")
```

Figure 39 – Table header for displaying execution times

To print each row in a clean and aligned format, the program uses the *printRow()* method (Guad, 2024).

```java
public static void printRow(String algorithm, long random, long sorted, long reversed) {
    System.out.printf("%-23s | %-11d | %-11d | %-13d%n",
            algorithm, random, sorted, reversed);
}
```

Figure 40 – PrintRow method used to format output in aligned columns

Here the *printf* us used to control how the output is formatted. The different % values decide how much space each column gets and make sure the text is aligned properly. This ensures that the columns are lined up, so the table becomes easier to read and compare.

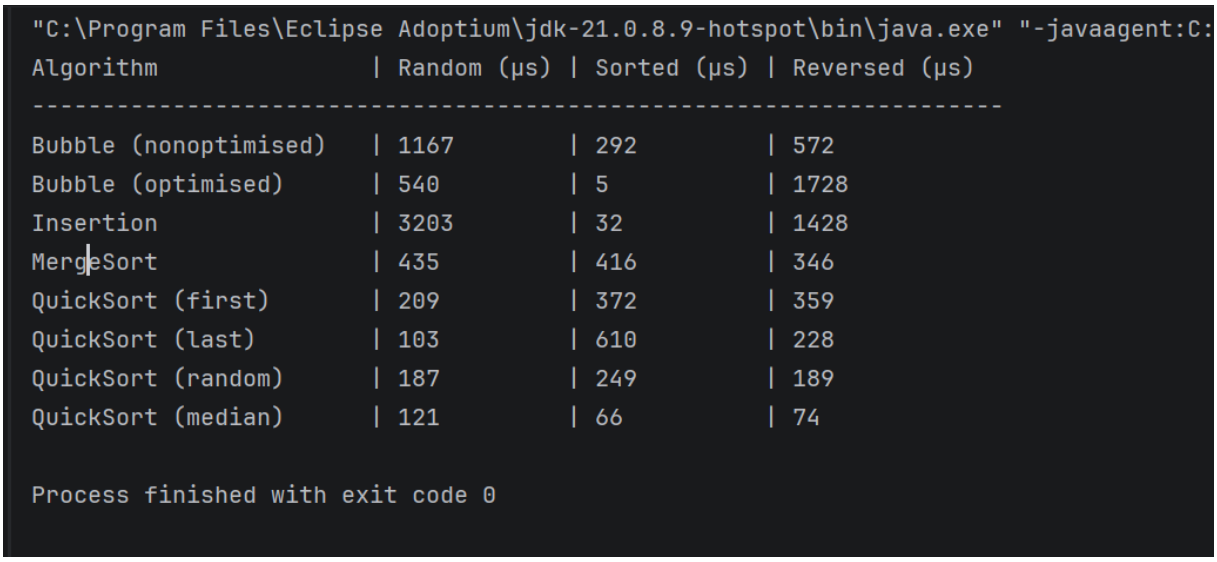


Figure 41 – Comparison of sorting algorithms across different input orders

Bubble Sort and Insertion Sort are strongly affected by input of the order. They perform very well when the data is already sorted but become much slower when the data is reversed. Merge Sort has almost the same execution time for all three cases. This shows that it is stable and not heavily influenced by input order. QuickSort performs differently depending on pivot choice. The random and median-of-three versions give more consistent results, while first and last element pivots can be slower depending on the input. Overall, the fastest and most reliable results come from Merge Sort and QuickSort when good pivot strategies are used.

8. Conclusion

This project successfully implemented and compared four fundamental sorting algorithms using alcohol content data from the Wine Quality dataset. The key findings are –

Bubble Sort and Insertion Sort perform well on small or nearly-sorted datasets but degrade to O(n^2) on larger random data, making them unsuitable for production use.

Merge Sort demonstrates consistent O(n log n) performance regardless of input order, with identical merge operation counts for both original and shuffled data. This predictability makes it ideal for large datasets despite O(n) extra memory usage.

QuickSort shows that pivot selection significantly impacts practical performance. The median-of-three strategy produced the fewest comparisons (640) compared to first-element (709), last-element (747, and random pivot (795), creating more balanced partitions.

Overall, advanced algorithms like Merge Sort and QuickSort (median-of-three pivot) clearly outperform simple algorithms for real-world datasets. The choice of algorithm depends on data characteristics, memory constraints and stability requirements, demonstrating the practical importance of algorithm analysis.

9. Reference List

Alake, R. (2025). *Bubble sort time complexity and algorithm explained*. https://builtin.com/data-science/bubble-sort-time-complexity

Cortez, P., Cerdeira, A., Almeida, F. Matos, T., & Reis, J. (2009). *Wine quality [Dataset]*. https://archive.ics.uci.edu/dataset/186/wine+quality

GeeksforGeeks. (2025a). *Shuffle a given array using Fisher-Yates shuffle algorithm*. https://www.geeksforgeeks.org/dsa/shuffle-a-given-array-using-fisher-yates-shuffle-algorithm/

GeeksforGeeks. (2025c). *Implement QuickSort with first element as pivot*. https://www.geeksforgeeks.org/dsa/implement-quicksort-with-first-element-as-pivot/

GeeksforGeeks. (2026b). *Insertion Sort Algorithm*. Insertion Sort Algorithm - GeeksforGeeks

Guad, A. (2024). *Format output in a Table Format Using System.out.* https://www.baeldung.com/java-format-output-table-system-out

Oracle. (n.d.-a). *Definition of the java programming language*. https://docs.oracle.com/javase/tutorial/getStarted/intro/definition.html

Oracle. (n.d.-b). *Object-oriented programming concepts*. https://docs.oracle.com/javase/tutorial/java/concepts/index.html

Oracle. (n.d.-c). *Java platform, standard Edition documentation*. https://docs.oracle.com/javase/8/docs/api/

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed). Addison-Wesley.